

Guia Técnico: Normalização de Dados com Casos Práticos

A **normalização de dados** é um processo formal e sistemático aplicado ao modelo relacional de bancos de dados. Seu principal objetivo é organizar as tabelas para **eliminar redundâncias**, **evitar anomalias de atualização** (inserção, alteração e exclusão) e **garantir a integridade referencial**.

Este documento explica os conceitos fundamentais da normalização por meio de um estudo de caso prático, utilizando as seguintes entidades de um sistema comercial: `CIDADE`, `CLIENTE`, `VENDA`, `VENDA\_ITENS` e `PRODUTO`.

1. Visão Geral das Formas Normais

O processo de normalização ocorre em estágios chamados **Formas Normais (FN)**. Cada forma normal possui regras específicas e cumulativas, ou seja, para uma tabela estar na 2ª Forma Normal, ela obrigatoriamente já deve cumprir os requisitos da 1ª Forma Normal.

- * **1ª Forma Normal (1FN):** Trata da atomicidade dos dados (valores únicos nas colunas) e eliminação de grupos repetitivos.
- * **2ª Forma Normal (2FN):** Elimina dependências parciais de chaves primárias compostas.
- * **3ª Forma Normal (3FN):** Elimina dependências transitivas (atributos não-chave que dependem de outros atributos não-chave).

2. Aplicação Prática no Estudo de Caso

Para ilustrar o processo de evolução do banco de dados, partiremos de uma estrutura conceitual não normalizada (0FN) até a estrutura perfeitamente otimizada apresentada no script SQL.

Cenário Inicial: Tabela Única Não Normalizada (0FN)

Imagine que toda a venda fosse registrada em uma única planilha ou tabela com a seguinte estrutura:

```
`TABELA_GERAL` (CodVenda, DataEmi, CpfCliente, NomeCliente, EnderecoCliente, CodCidade, NomeCidade, UfCidade, TotalVenda, CodProduto, DescricaoProduto, VlUnitario, Quantidade, VlTotalItem)`
```

Neste cenário simplista, se o cliente comprasse 10 produtos diferentes na mesma venda, os dados do cliente e da cidade se repetiriam 10 vezes. Isso gera desperdício de espaço e o risco de inconsistências (ex: alterar a cidade de um cliente em uma linha da venda e esquecer de alterar nas outras).

Primeira Forma Normal (1FN)

Regra Principal: Todos os atributos devem ser atômicos (não divisíveis) e não devem existir grupos repetitivos ou vetores em uma mesma linha.

****Aplicações no modelo:****

1. ****Separação dos Itens da Venda:**** O grupo de produtos de uma venda constitui um grupo repetitivo. Para resolver, isolamos a parte do cabeçalho da venda da parte dos itens.
2. ****Atributos Atômicos:**** Garantir que nomes e endereços não estejam agrupados em uma única string confusa.

****Resultado da 1FN:****

```
* `VENDA(VEN_CODIGO, VEN_DT_EMI, VEN_VL_TOTAL, CLI_CODIGO, CLI_NOME,
CLI_ENDERECO, CID_CODIGO, CID_NOME, CID_UF)`
* `VENDA_ITENS(VEN_CODIGO, PRD_CODIGO, PRD_DESCRICAO, VL_UNIT, QT,
VL_TOTAL)`
```

> ****Nota:**** Embora tenhamos separado os itens, a estrutura ainda possui problemas graves de redundância que as próximas formas normais irão tratar.

Segunda Forma Normal (2FN)

****Regra Principal:**** Estar na 1FN e garantir que todos os atributos não-chave dependam ****totalmente**** da chave primária (Eliminação da Dependência Parcial). Esta regra se aplica principalmente a tabelas com chaves primárias compostas.

****Aplicações no modelo (Foco na tabela `VENDA\_ITENS`):****

Na 1FN, a tabela `VENDA\_ITENS` possui uma chave primária composta por `(VEN\_CODIGO, PRD\_CODIGO)`.

* O atributo `QT` (Quantidade) depende tanto da venda quanto do produto para existir.

* Porém, o atributo `PRD\_DESCRICAO` depende ****apenas**** do `PRD\_CODIGO`. Ele não muda de acordo com o número da venda. Isso é uma ***dependência parcial***.

****Resolução:****

Removemos `PRD\_DESCRICAO` de `VENDA\_ITENS` e criamos uma tabela própria para os produtos.

****Resultado da 2FN:****

```
* `PRODUTO(PRD_CODIGO, PRD_DESCRICAO, PRD_VALOR, PRD_QT)`
* `VENDA_ITENS(VEN_CODIGO, PRD_CODIGO, VL_UNIT, QT, VL_TOTAL)` *(Agora
`VL_UNIT`, `QT` e `VL_TOTAL` dependem puramente da combinação da chave
composta).*
```

Terceira Forma Normal (3FN)

****Regra Principal:**** Estar na 2FN e garantir que nenhum atributo não-chave dependa de outro atributo não-chave (Eliminação da Dependência Transitiva). Em termos práticos: X determina Y, e Y determina Z, logo X determina Z por transição.

****Aplicações no modelo (Foco na tabela `VENDA`):****

A estrutura temporária da tabela `VENDA` possuía dados do cliente e da cidade vinculados diretamente ao código da venda.

1. O `VEN\_CODIGO` (Chave) define o `CLI\_CODIGO`.
2. O `CLI\_CODIGO` (Não-chave) define o `CLI\_NOME`.
3. Portanto, o nome do cliente está dependendo indiretamente (transitivamente) do código da venda.

Mesmo problema ocorre com a cidade:

1. O `CLI\_CODIGO` define o `CID\_CODIGO`.
2. O `CID\_CODIGO` define o `CID\_NOME` e `CID\_UF`.

****Resolução:****

Extraímos os dados dos clientes para a tabela `CLIENTE` e os dados geográficos para a tabela `CIDADE`, mantendo apenas os códigos (chaves estrangeiras) para fazer as conexões.

****Resultado Final Perfeitamente Normalizado (3FN):****

```
*  **`CIDADE`**: Guarda apenas o cadastro geográfico.
*    *Chave:* `CID_CODIGO`
*  **`CLIENTE`**: Depende apenas da chave do cliente e referencia a
cidade via FK.
*    *Chave:* `CLI_CODIGO` | *FK:* `CID_CODIGO`
*  **`PRODUTO`**: Cadastro puro dos itens de estoque.
*    *Chave:* `PRD_CODIGO`
*  **`VENDA`**: Contém os dados globais do cupom/nota e referencia o
cliente.
*    *Chave:* `VEN_CODIGO` | *FK:* `CLI_CODIGO`
*  **`VENDA_ITENS`**: Relacionamento N:N entre Venda e Produto.
*    *Chave Composta:* `(PRD_CODIGO, VEN_CODIGO)` | *FKs:*
`PRD_CODIGO`, `VEN_CODIGO`
```

3. Conclusão e Benefícios do Modelo Proposto

Ao finalizar a 3FN, o modelo de banco de dados atinge estabilidade operacional absoluta:

- * ****Consistência:**** Se a cidade "Maringá" mudar de nome ou UF, altera-se apenas um registro na tabela `CIDADE`, propagando instantaneamente para todos os clientes.
- * ****Performance e Armazenamento:**** Dados textuais longos (nomes, descrições, endereços) não são duplicados no histórico de movimentações (`VENDA` e `VENDA\_ITENS`), reduzindo drasticamente o tamanho físico do banco de dados.
- * ****Integridade:**** As chaves estrangeiras impedem que uma venda seja registrada para um cliente inexistente ou que um item seja vendido sem um código de produto válido.